

# ECE0202 Lecture 14: Interrupts 2

## 中文逐页详细讲义

主题：Interrupt Number、Enable/Disable、Priority and Preemption、Mask、SysTick

**本讲主线** Lecture 13 讲的是“一个中断怎样进入 ISR、怎样 stacking/unstacking”。Lecture 14 继续往下讲：**多个中断怎样编号、怎样打开/关闭、谁先执行、谁能打断谁、怎样用 SysTick 定时产生中断**。如果把 CPU 当成做题的人，NVIC 就是门口的调度员；SysTick 是一个固定间隔敲门的闹钟。

## 目录

|                                                                    |           |
|--------------------------------------------------------------------|-----------|
| <b>0. 先记住的核心词汇</b>                                                 | <b>2</b>  |
| <b>1 逐页解释</b>                                                      | <b>3</b>  |
| Slide 1 标题页：Interrupts 2                                           | 3         |
| Slide 2 Topics：本讲主题                                                | 3         |
| Slides 3–4 Interrupt Number 与 CMSIS Interrupt Number               | 3         |
| Slide 5 Enable an Interrupt：中断使能                                   | 3         |
| Slides 6–7 Enabling / Disabling Peripheral Interrupts：打开和关闭外设中断    | 3         |
| Slide 8 Interrupt Priority：优先级的反直觉规则                               | 4         |
| Slide 9 Priority Register：抢占优先级与子优先级                               | 4         |
| Slide 10 NVIC_SetPriorityGrouping：优先级位如何切分                         | 4         |
| Slides 11–12 Priority of System Interrupts / Peripheral Interrupts | 5         |
| Slides 13–19 Nested Interrupts: Example of Preemption：嵌套中断抢占例子     | 5         |
| Slide 20 Tail Chaining：尾链优化                                        | 6         |
| Slides 21–24 Masking Priority 与 PRIMASK / FAULTMASK / BASEPRI      | 6         |
| Slides 25–26 System Timer (SysTick)：系统滴答定时器                        | 7         |
| Slides 27–30 SysTick 工作原理图：Reload、Counter、COUNTFLAG、TICKINT、ENABLE | 7         |
| Slide 31 Registers of System Timer：SysTick 四个寄存器总览                 | 8         |
| Slide 32 SysTick_LOAD：重装载值寄存器                                      | 8         |
| Slide 33 SysTick_VAL：当前值寄存器                                        | 8         |
| Slide 34 SysTick_CALIB：校准寄存器                                       | 8         |
| Slide 35 Calculating Reload Value：计算重装载值                           | 9         |
| Slides 36–39 Implementing Delay Function：用 SysTick 实现 delay        | 9         |
| Slides 40–41 systick_init：SysTick 初始化代码逐行解释                        | 10        |
| <b>2 本讲最终总结</b>                                                    | <b>12</b> |
| <b>3 考前速记表</b>                                                     | <b>12</b> |

## 0. 先记住的核心词汇

| 英文/缩写                | 中文         | 直白解释                                                 |
|----------------------|------------|------------------------------------------------------|
| Interrupt            | 中断         | 外设或系统事件打断当前程序，让 CPU 先去处理事件。                          |
| ISR / Handler        | 中断服务程序     | 中断发生后 CPU 跳去执行的函数，例如 SysTick_Handler。                |
| NVIC                 | 嵌套向量中断控制器  | Cortex-M 内核里的中断调度器，管 enable、pending、active、priority。 |
| IRQn                 | 中断请求编号     | CMSIS 里用来标识某个中断的编号。系统异常多为负数，外设中断从 0 开始。              |
| System Exception     | 系统异常       | ARM 内核自己定义的异常，例如 NMI、HardFault、SysTick。              |
| Peripheral Interrupt | 外设中断       | 芯片厂家定义的外设事件中断，例如 EXTI、TIM、DMA、UART。                  |
| Enable Register      | 使能寄存器      | 决定某个外设中断是否被允许进入 NVIC。                                |
| Pending Register     | 挂起寄存器      | 表示某个中断已经发生，但还在排队等待处理。                                |
| Active Register      | 活跃寄存器      | 表示某个中断正在被 CPU 执行。                                    |
| Priority Register    | 优先级寄存器     | 决定多个中断同时出现时谁更急。注意：数字越小越急。                            |
| Preemption           | 抢占         | 高优先级中断打断低优先级 ISR。                                    |
| Tail Chaining        | 尾链         | 一个 ISR 结束后直接接下一个 ISR，省掉一次完整 unstacking + stacking。   |
| PRIMASK              | 优先级屏蔽寄存器之一 | 一键屏蔽大多数可配置中断，HardFault 和 NMI 不受影响。                   |
| FAULTMASK            | 故障屏蔽寄存器    | 屏蔽几乎所有异常，只留下 NMI。                                    |
| BASEPRI              | 基准优先级寄存器   | 只屏蔽“紧急程度低于某阈值”的中断。                                   |
| SysTick              | 系统滴答定时器    | Cortex-M 内置的 24-bit 倒数计数定时器，可周期性产生 SysTick 中断。       |

# 1 逐页解释

## Slide 1 标题页：Interrupts 2

这一页只是封面。真正内容从 Slide 2 开始。Lecture 14 是 Lecture 13 的延续：Lecture 13 解决“中断怎么进、怎么回”；Lecture 14 解决“中断很多时怎么管理”。

你必须把本讲看成四个问题：

1. 这个中断叫什么？对应 Interrupt Number / IRQn。
2. 这个中断能不能进来？对应 Enable / Disable。
3. 多个中断谁先执行？对应 Priority / Preemption。
4. 某段关键代码能不能临时挡住中断？对应 Mask。

## Slide 2 Topics：本讲主题

本页列出四个主题：Interrupt Number (中断编号)、Enable/Disable (使能/禁止)、Priority and Preemption (优先级与抢占)、Mask (中断屏蔽)。

**学习顺序** 先知道中断编号，再知道如何打开，再知道谁先执行，最后知道如何临时屏蔽。顺序不能乱。你如果连一个中断的编号都不知道，就谈不上设置优先级或使能。

## Slides 3–4 Interrupt Number 与 CMSIS Interrupt Number

**核心：每个中断都必须有编号。NVIC 靠编号识别中断，不靠人的名字识别。**

Cortex-M 支持最多 256 个中断/异常入口。课程这里把它们分成两类：

- **System Exceptions** (系统异常)：前 16 个，由 ARM 内核定义。CMSIS 里通常用负数表示，例如 SysTick\_IRQn = -1、HardFault\_IRQn = -13、NonMaskableInt\_IRQn = -14。
- **Peripheral Interrupts** (外设中断)：其余最多 240 个，由芯片厂家定义。编号从 0 开始，例如 WWDG\_IRQn = 0、EXTI0\_IRQn = 6。

**CMSIS** (Cortex Microcontroller Software Interface Standard, Cortex 微控制器软件接口标准) 的作用是把些中断编号写成统一的名字。比如你不需要死记某个中断是几号，只要用 TIM7\_IRQn、SysTick\_IRQn 这种名字。

**易错点** “负数中断号”不代表优先级一定低，也不是内存地址。它只是 CMSIS 为了区分系统异常和外设中断而使用的编号方式。外设中断从 0 开始，系统异常用负数。

## Slide 5 Enable an Interrupt：中断使能

**使能**就是“允许这个中断被响应”。外设事件发生和 CPU 真正进入 ISR 是两件事，中间隔着 NVIC。

系统异常和外设中断的使能方式不同：

- **System exception** (系统异常)：有些永远打开，不能关闭，比如 NMI、HardFault；有些由自己的模块控制，比如 SysTick 由 SysTick 模块自己的 CTRL 寄存器控制。
- **Peripheral interrupt** (外设中断)：通常由 NVIC 集中管理，使用 **ISER** (Interrupt Set-Enable Register, 使能寄存器) 打开，使用 **ICER** (Interrupt Clear-Enable Register, 清除使能寄存器) 关闭。

直白理解：外设说“我有事”，NVIC 会先看“这个中断开门了吗”。如果没使能，事件可能发生了，但 CPU 不会跳去 ISR。

## Slides 6–7 Enabling / Disabling Peripheral Interrupts：打开和关闭外设中断

以 TIM7\_IRQn = 55 为例：

```
LDR R0, =TIM7_IRQn
BL  NVIC_EnableIRQ      ; Enable Timer7 interrupt

LDR R0, =TIM7_IRQn
BL  NVIC_DisableIRQ     ; Disable Timer7 interrupt
```

逐行解释：

- LDR R0, =TIM7\_IRQn：把 TIM7 的中断编号放进 R0。按照 ARM 调用约定，R0 通常用来传第一个函数参数。

- BL NVIC\_EnableIRQ: 调用 CMSIS 提供的函数, 告诉 NVIC 允许 TIM7 中断。
- BL NVIC\_DisableIRQ: 调用 CMSIS 提供的函数, 告诉 NVIC 禁止 TIM7 中断。

**关键逻辑** NVIC\_EnableIRQ 不是让 TIM7 开始计数。它只是让 “TIM7 产生中断请求时, NVIC 允许 CPU 响应”。定时器本身是否工作, 要看 TIM7 自己的寄存器配置。

## Slide 8 Interrupt Priority: 优先级的反直觉规则

Cortex-M 的优先级有一个反直觉规则:

**优先级数值越小, 紧急程度越高。**

例如:

- Interrupt A priority = 5
- Interrupt B priority = 2

结论不是 A 更高, 而是 **B 更高**, 因为 2 比 5 更小。

Reset、NMI、HardFault 的优先级固定, 不能随便改:

- Reset: 最高, priority = -3。
- NMI: 第二高, priority = -2。
- HardFault: 第三高, priority = -1。

其他大多数中断和异常可以设置优先级。

**考试陷阱** “priority value 高” 不等于 “优先级高”。在这里, 数字大通常意味着更不急。看到 priority = 2 和 priority = 5, 必须立刻判断 2 更急。

## Slide 9 Priority Register: 抢占优先级与子优先级

每个可配置中断的 priority 不只是一个单纯数字, 它可以被拆成两个部分:

- **Preempt priority number** (抢占优先级编号): 决定一个中断能不能打断当前正在执行的 ISR。
  - **Sub-priority number** (子优先级编号): 当多个中断已经 pending, 而且抢占优先级相同, 它决定谁先执行。
- 更直白:

- Preempt priority 管 “能不能插队”。
- Sub-priority 管 “同一档里面谁排前面”。

例子: 如果 A 和 B 的 preempt priority 不同, 更紧急的那个可以抢占; 如果 preempt priority 相同, 它们不能互相抢占, 只能等当前 ISR 结束, 再由 sub-priority 决定下一个执行谁。

## Slide 10 NVIC\_SetPriorityGrouping: 优先级位如何切分

STM32L4 常见可用 priority 位数是 4 bit。NVIC\_SetPriorityGrouping(n) 决定这 4 bit 如何分给 preemption 和 sub-priority。

| n    | 抢占优先级位数 | 子优先级位数 |
|------|---------|--------|
| 0    | 0       | 4      |
| 1    | 1       | 3      |
| 2 默认 | 2       | 2      |
| 3    | 3       | 1      |
| 4    | 4       | 0      |

**怎么理解这张表:**

- 抢占优先级位数越多, 可分的 “紧急等级” 越多, 中断之间越容易形成抢占关系。
- 子优先级位数越多, 同一抢占等级下的排队顺序越细。
- 默认 n=2, 就是 2 bit 管抢占, 2 bit 管同级排队。

**不要过度脑补** 这不是改变中断总数, 也不是改变 ISR 地址。它只是改变 priority 字段内部的解释方式。

## Slides 11–12 Priority of System Interrupts / Peripheral Interrupts

这两页讲如何给系统异常或外设中断设置优先级。Slides 11 的代码写法是：

```
LDR R0, =TIM7_IRQn      ; which interrupt
MOV R1, #0               ; priority
BL NVIC_SetPriority      ; set priority level
```

解释：

- R0：传入要设置的中断编号。
- R1：传入优先级数值。
- NVIC\_SetPriority：CMSIS 函数，会根据 IRQn 是负数还是非负数，去写对应的优先级寄存器。  
系统异常和外设中断底层寄存器不同：
- 系统异常，如 SysTick，优先级通常在 SCB（System Control Block，系统控制块）的 SHP 相关寄存器里。
- 外设中断，如 TIM7、EXTI，优先级在 NVIC IP（Interrupt Priority，中断优先级数组）里。  
你写 CMSIS 函数时可以暂时不用手算这些地址，但必须知道背后本质是：**写某个 priority register 的某几个 bit。**

## Slides 13–19 Nested Interrupts: Example of Preemption：嵌套中断抢占例子

这组页是本讲最核心内容之一。它演示：一个中断 ISR 正在执行时，另一个更高优先级中断来了，会发生什么。

### 初始状态：EXTI3 先发生

图中 EXTI3 对应 interrupt number 9，DMA1\_Channel2 对应 interrupt number 12。表中：

- Enable Register：9 和 12 都是 1，说明 EXTI3 和 DMA1\_Channel2 都允许进入 NVIC。
- Priority Register：EXTI3 的 priority = 5，DMA1\_Channel2 的 priority = 3。
- 因为 3 比 5 小，所以 DMA1\_Channel2 更急。

### 第一步：EXTI3 进入 ISR 9

EXTI3 先发生，CPU 先进行 stacking，把主程序现场保存到栈里，然后进入 ISR 9。此时：

- EXTI3 的 Active Register 变成 1，表示 ISR 9 正在执行。
- 栈里有一份主程序现场：R0、R1、R2、R3、R12、LR、PC、xPSR。

### 第二步：DMA1\_Channel2 在 ISR 9 执行期间发生

当 ISR 9 正在执行时，DMA1\_Channel2 中断来了。NVIC 会比较两者优先级：

DMA1\_Channel2 priority = 3    vs    EXTI3 priority = 5

因为 3 更小，所以 DMA1\_Channel2 更紧急，可以抢占 EXTI3。

### 第三步：发生第二次 stacking，进入 ISR 12

DMA1\_Channel2 抢占 ISR 9 时，CPU 不是直接跳过去。它必须先保存 ISR 9 当前现场，所以又做一次 stacking。此时栈中有两层：

- 第一层：主程序被 EXTI3 打断时的现场。
- 第二层：ISR 9 被 DMA1\_Channel2 打断时的现场。

然后 CPU 进入 ISR 12，DMA1\_Channel2 的 Active 变成 1。

### 第四步：ISR 12 结束后回到 ISR 9

DMA1\_Channel2 的 ISR 12 执行完后，执行异常返回。CPU unstacking 第二层栈帧，恢复 ISR 9 的现场，于是回到 ISR 9 继续执行。

### 第五步：ISR 9 结束后回到主程序

ISR 9 执行完后，再 unstacking 第一层栈帧，恢复主程序现场，回到原来的主程序。

**这组图的本质** 抢占不是“两个 ISR 同时执行”。Cortex-M 单核一次只能执行一条指令。抢占的本质是：高优先级中断暂时打断低优先级 ISR，低优先级 ISR 的现场被压栈保存，等高优先级 ISR 结束后再恢复。

**易错点** Pending 和 Active 不是一回事。Pending 表示“来了但还没处理”；Active 表示“正在处理”。一个中断可能先 pending，之后进入 active；也可能因为优先级更高而直接抢占进入 active。

### Slide 20 Tail Chaining：尾链优化

Tail Chaining 是 Cortex-M 对连续中断的硬件优化。

普通情况：

1. EXTI3 进入 ISR 9，开始 stacking。
2. ISR 9 结束，unstacking 回主程序。
3. 发现 EXTI4 pending，再 stacking 进入 ISR 10。

这样会浪费时间：先恢复主程序现场，马上又保存主程序现场。

Tail Chaining 的优化是：

**ISR 9 刚结束，如果发现还有 pending 的 ISR 10，就不先回主程序，而是直接接到 ISR 10。**

图中标出：正常 unstacking + stacking 可能各 12 cycles，而 tail chaining 只需要约 6 cycles。意思是硬件节省了切换成本。

**通俗记忆** Tail Chaining 就像接力跑。第一个 ISR 跑完，不必先回到主程序“休息区”，直接把控制权交给下一个 ISR。

### Slides 21–24 Masking Priority 与 PRIMASK / FAULTMASK / BASEPRI

Masking 就是“临时挡住一部分中断”。用途是保护 critical code（临界代码/关键代码），避免某些操作做到一半被中断打断。

#### 为什么需要 mask？

假设你正在修改一个共享变量，需要两步：

```
LDR r0, [counter]
ADD r0, r0, #1
STR r0, [counter]
```

如果中间被 ISR 打断，而 ISR 也改 counter，就可能出现数据错乱。解决方法之一就是在关键区间临时屏蔽部分中断。

#### 三个屏蔽寄存器

| 寄存器       | 作用                                                   |
|-----------|------------------------------------------------------|
| FAULTMASK | 最狠。写 1 后屏蔽所有异常/中断，除了 NMI。一般不轻易用。                     |
| PRIMASK   | 写 1 后屏蔽所有可配置异常/中断，但 NMI 和 HardFault 仍可进来。常用于非常短的临界区。 |
| BASEPRI   | 更精细。设置一个优先级阈值，屏蔽“紧急程度低于或等于阈值”的中断，更紧急的中断仍允许进来。        |

#### 示例代码解释

```
MOV R0, #1
MSR FAULTMASK, R0 ; disable all interrupts except NMI

MOV R0, #0
```

```
MSR FAULTMASK, R0 ; enable interrupts again
```

```
MOV R0, #0x60
```

```
MSR BASEPRI, R0 ; mask interrupts with encoded priority >= 0x60
```

MSR 的意思是 Move to Special Register (把通用寄存器的值写入特殊寄存器)。这里把 R0 的值写入 FAULTMASK 或 BASEPRI。

**关键陷阱** BASEPRI 不是“屏蔽所有中断”。它只屏蔽优先级数值不够紧急的一部分中断。因为数字越小越紧急，所以 BASEPRI=0x60 通常允许数值小于 0x60 的更高紧急级别中断继续进入。

## Slides 25–26 System Timer (SysTick): 系统滴答定时器

SysTick (System Timer, 系统滴答定时器) 是 ARM Cortex-M 内核自带的标准硬件，不是 STM32 厂商额外加的普通 TIM 外设。

它的作用：

- 按固定时间间隔产生 SysTick 中断。
- 用来测量经过时间，例如 delay 函数。
- 用来周期性执行任务，例如操作系统调度、周期轮询。

SysTick 的中断服务函数通常叫：

```
SysTick_Handler PROC
    BX LR
ENDP
```

真正使用时，里面不会只有 BX LR，而是会更新某个全局变量、计数器或任务标志。

**本质** SysTick 是一个自动倒计时闹钟。每次数到 0，就向 NVIC 发出 SysTick interrupt，CPU 进入 SysTick\_Handler。

## Slides 27–30 SysTick 工作原理图: Reload、Counter、COUNTFLAG、TICKINT、ENABLE

SysTick 的核心结构很简单：

- 一个 Reload Value (重装载值)。
- 一个 24-bit down counter (24 位向下计数器)。
- 计数器从 Reload 数到 0。
- 到 0 后设置 COUNTFLAG，如果中断允许，就产生 SysTick interrupt。

几个关键 bit

| 字段        | 中文      | 作用                                             |
|-----------|---------|------------------------------------------------|
| ENABLE    | 使能位     | 1 表示 SysTick 定时器开始工作；0 表示停止。                   |
| TICKINT   | 滴答中断使能位 | 1 表示计数到 0 时产生 SysTick interrupt；0 表示只计数不进 ISR。 |
| CLKSOURCE | 时钟源选择位  | 选择处理器时钟或外部参考时钟作为计数输入。                          |
| COUNTFLAG | 计数完成标志  | 计数器从 1 变到 0 后置 1，表示一次计时周期结束。                   |

### 时钟源选择

图里有 processor clock 和 external clock。一般理解：

- processor clock：用 CPU/AHB 相关时钟，速度高。
- external clock：外部参考时钟或经过分频的时钟，可能用于更稳定或特定频率。

**不要混淆** ENABLE=1 只是让 SysTick 开始计数；TICKINT=1 才表示计数到 0 后会触发中断。只开 ENABLE 不开 TICKINT，可能只是定时器在跑，但不会进 SysTick\_Handler。



## Slide 31 Registers of System Timer: SysTick 四个寄存器总览

SysTick 主要有四个寄存器：

| 寄存器           | 中文       | 作用                                |
|---------------|----------|-----------------------------------|
| SysTick_CTRL  | 控制与状态寄存器 | 控制是否启用、是否产生中断、时钟源, 读取 COUNT-FLAG。 |
| SysTick_LOAD  | 重装载值寄存器  | 决定每次中断间隔是多少个时钟周期。                 |
| SysTick_VAL   | 当前值寄存器   | 读取当前倒计数值；写入可清零当前计数器。              |
| SysTick_CALIB | 校准寄存器    | 提供参考校准值，如 TENMS、SKEW、NOREF。       |

这四个寄存器是 memory-mapped registers。也就是说，它们在地址空间里有固定地址，CPU 通过 LDR/STR 读写它们。

## Slide 32 SysTick\_LOAD: 重装载值寄存器

SysTick\_LOAD 是 24-bit，最大值：

$$0x00FFFFFF = 16,777,215$$

计数器从 RELOAD 往下数到 0。因此两个 SysTick 中断之间的间隔是：

$$\text{Interval} = (\text{RELOAD} + 1) \times \text{Source Clock Period}$$

为什么是 RELOAD + 1，不是 RELOAD？因为如果 RELOAD=6，计数序列是：

$$6, 5, 4, 3, 2, 1, 0$$

一共 7 个状态，也就是 6+1 个时钟周期。

**最常用公式** 如果你想要 N 个时钟周期产生一次中断，就设置  $\text{RELOAD} = N - 1$ 。

## Slide 33 SysTick\_VAL: 当前值寄存器

SysTick\_VAL 保存当前倒计数的数值。

它有几个重要性质：

- 读取它，可以看到当前 counter 数到哪里。
- 当 counter 从 1 变成 0，会产生一次计时完成事件，并可能触发中断。
- 写入 SysTick\_VAL 会清零 counter 和 COUNTFLAG。
- 写 SysTick\_VAL 不会立刻触发 SysTick 中断，只是让下一次 timer clock 时重新装载。
- reset 后 VAL 可能是随机值，所以启用 SysTick 前应该先清零。

**易错点** 初始化 SysTick 时必须清 VAL。否则第一次中断间隔可能不是你想象的完整周期，因为计数器初值可能不是干净的。

## Slide 34 SysTick\_CALIB: 校准寄存器

SysTick\_CALIB 是 read-only，不能随便写。它提供芯片设计者给出的校准信息：

- TENMS：理论上给出产生 10 ms 周期所需的 reload 值。
- SKEW：表示校准值是否精确。
- NOREF：表示是否有外部参考时钟。

课件特别提醒：这个寄存器可能没有实现，或者不同芯片定义不同。比如 STM32L4 中该字段可能对应 1 ms 的 reload 值，而不是严格的 10 ms。

**实用判断** 真做题或实验时，最可靠的是用时钟频率自己算 LOAD，不要盲信 CALIB。



## Slide 35 Calculating Reload Value: 计算重装载值

目标: 时钟源为 80 MHz, 希望 SysTick 每 10 ms 中断一次。

公式:

$$\begin{aligned}\text{Reload} &= \frac{\text{Target Interval}}{\text{Clock Period}} - 1 \\ &= \text{Target Interval} \times \text{Clock Frequency} - 1\end{aligned}$$

代入:

$$\begin{aligned}\text{Reload} &= 10 \text{ ms} \times 80 \text{ MHz} - 1 \\ &= 10 \times 10^{-3} \times 80 \times 10^6 - 1 \\ &= 800000 - 1 \\ &= 799999\end{aligned}$$

所以应该写:

$$\text{SysTick\_LOAD} = 799999$$

检查是否超范围: 799999 小于 16,777,215, 所以 24-bit SysTick 可以装下。

## Slides 36–39 Implementing Delay Function: 用 SysTick 实现 delay

这组页把 SysTick 应用到 delay 函数。目标是每次 delay 1 second。

### 整体思路

1. 初始化 SysTick, 让它每 1 ms 产生一次中断。
2. delay 函数把全局变量 TimeDelay 设置为 1000。
3. 每次 SysTick 中断发生, SysTick\_Handler 把 TimeDelay 减 1。
4. delay 函数一直检查 TimeDelay 是否变成 0。
5. 1000 次 1 ms 后, 约等于 1 s, delay 结束。

### 主函数

```
TimeDelay DCD 0x00 ; global, volatile variable

__main PROC
    ...
    LDR r0, =1000 ; Interrupt period = 1000 cycles
    BL systick_init
    ...
    LDR r0, =1000 ; Delay 1000 interrupts
    BL delay
    ...
ENDP
```

TimeDelay 是全局变量, 放在内存中。这里必须强调 volatile (易变变量) 的意义: 这个变量会被 ISR 改变, 主程序不能假设它不会自己变。

### delay 函数

```
delay PROC
    LDR r1, =TimeDelay
    STR r0, [r1] ; r0 specifies the delay time length

delayloop
    LDR r1, =TimeDelay
    LDR r2, [r1]
    CMP r2, #0
```

```
BGT delayloop      ; Busy wait
BX LR
ENDP
```

解释:

- STR r0, [r1]: 把 delay 的目标次数写入 TimeDelay。
- CMP r2, #0: 检查 TimeDelay 是否已经减到 0。
- BGT delayloop: 如果还大于 0, 就继续等。
- 这叫 busy wait (忙等待): CPU 一直反复检查变量, 没有去做别的事。

### SysTick\_Handler

```
SysTick_Handler PROC ; SysTick interrupt service routine
    LDR r1, =TimeDelay ; TimeDelay is a global volatile variable
    LDR r2, [r1]
    CMP r2, #0
    SUBGT r2, r2, #1 ; Prevent it from being negative
    STR r2, [r1]
    BX LR
ENDP
```

解释:

- 每次 SysTick 中断进来, ISR 都会执行一次。
- 它读出 TimeDelay。
- 如果 TimeDelay > 0, 就减 1。
- 再写回内存。
- BX LR 表示 ISR 结束, 触发异常返回, 恢复现场。

**核心理解** delay 函数和 SysTick\_Handler 是“配合工作”的: delay 在主程序里等变量变 0; SysTick\_Handler 在中断里周期性把变量减 1。

## Slides 40–41 systick\_init: SysTick 初始化代码逐行解释

这两页是完整初始化过程, 必须按顺序看。

### 第一部分: 关闭 SysTick

```
systick_init PROC
    LDR r1, =SysTick
    LDR r2, =0x0
    STR r2, [r1, #0] ; offset of SysTick_CTRL is 0
```

先把 SysTick\_CTRL 清零, 意思是先关掉 SysTick, 避免配置过程中乱触发。

### 第二部分: 设置 LOAD

```
LDR r1, =SysTick
SUB r0, r0, #1
STR r0, [r1, #4] ; offset of SysTick_LOAD is 4
```

r0 是传进来的目标周期数。因为公式是  $RELOAD = cycles - 1$ , 所以先 SUB r0, r0, #1, 再写到 SysTick\_LOAD。

### 第三部分: 清 VAL

```
LDR r1, =SysTick
LDR r2, =0x0
STR r2, [r1, #8] ; offset of SysTick_VAL is 8
```

清空当前计数器, 保证第一次计时从干净状态开始。

#### 第四部分：设置 SysTick 优先级

```
PUSH {LR, R0}
BL config_NVIC_in_C
POP {LR, R0}
```

BL 会改 LR，所以调用前先保存 LR。这里也保存 R0，是为了防止被调用函数改掉 R0。返回后再 POP 恢复。

#### 第五部分：打开 SysTick interrupt

```
LDR r1, =SysTick
LDR r2, [r1, #0]
ORR r2, #0x02
STR r2, [r1, #0]
```

0x02 对应 bit 1，也就是 TICKINT。这一步表示：计数到 0 时允许产生 SysTick 中断。

#### 第六部分：选择时钟源

```
LDR r1, =SysTick
LDR r2, [r1, #0]
BIC r2, #0x04
STR r2, [r1, #0]
```

0x04 对应 bit 2，也就是 CLKSOURCE。BIC r2, #0x04 表示清掉 bit 2，选择外部 1 MHz clock。BIC 可以理解为“用反掩码清位”。

#### 第七部分：真正启动 SysTick

```
LDR r1, =SysTick
LDR r2, [r1, #0]
ORR r2, #0x01
STR r2, [r1, #0]
BX LR
ENDP
```

0x01 对应 bit 0，也就是 ENABLE。这一步后 SysTick 才真正开始倒数计数。

**初始化顺序** 关 SysTick → 设置 LOAD → 清 VAL → 设置优先级 → 打开 TICKINT → 选时钟源 → 打开 ENABLE。  
不要先 enable 再配置，否则可能产生不可控的第一次中断。

## 2 本讲最终总结

1. 中断必须有编号。系统异常通常用负数  $IRQ_n$ ，外设中断从 0 开始。
2. NVIC 是中断调度器，管 enable、pending、active、priority。
3. 外设中断要进入 CPU，至少要外设产生请求、NVIC 使能、优先级允许。
4. 优先级数字越小，中断越紧急。
5. Preempt priority 决定能不能抢占；sub-priority 决定同级 pending 队列顺序。
6. 高优先级中断可以打断低优先级 ISR，形成 nested interrupt。
7. Tail chaining 是连续 ISR 之间的硬件优化，减少无意义的出栈入栈。
8. PRIMASK、FAULTMASK、BASEPRI 用于屏蔽中断，BASEPRI 最适合做“只屏蔽低紧急程度中断”的精细控制。
9. SysTick 是 Cortex-M 内置 24-bit 倒计数定时器，常用于周期中断和 delay。
10. SysTick 周期公式： $Interval = (RELOAD + 1) \times \text{Clock Period}$ 。
11. 用 SysTick 做 delay 的本质是：ISR 周期性修改全局变量，主程序 busy wait 等这个变量变成 0。

**最容易错的三句话** 第一，priority 数字越小越急。第二，enable 中断不等于外设已经工作。第三，SysTick 的 LOAD 要写“周期数 - 1”，不是直接写周期数。

## 3 考前速记表

| 问题                    | 直接答案                                       |
|-----------------------|--------------------------------------------|
| SysTick 是什么？          | Cortex-M 内置系统定时器，周期性产生 SysTick 中断。         |
| NVIC 是什么？             | 嵌套向量中断控制器，负责中断使能、挂起、优先级、抢占。                |
| $IRQ_n$ 负数是什么？        | 系统异常编号，例如 $SysTick = -1$ 。                 |
| $IRQ_n$ 非负数是什么？       | 外设中断编号，例如 $WWDG=0$ ， $EXTI0=6$ 。           |
| 优先级 2 和 5 谁更急？        | 2 更急。数字越小越高。                               |
| Preempt priority 管什么？ | 管能不能抢占当前 ISR。                              |
| Sub-priority 管什么？     | 管同一抢占等级下 pending 队列谁先执行。                   |
| Tail chaining 有什么用？   | 连续执行 ISR 时省掉不必要的 unstacking/stacking。      |
| PRIMASK 会屏蔽什么？        | 屏蔽大多数可配置中断，但不屏蔽 NMI 和 HardFault。           |
| BASEPRI 有什么特点？        | 按优先级阈值屏蔽低紧急程度中断。                           |
| SysTick_LOAD 最大多少？    | 24-bit 最大 $0x00FFFFFF$ 。                   |
| RELOAD 公式？            | $RELOAD = Interval \times f_{clock} - 1$ 。 |
| 为什么先清 VAL？            | 避免 reset 后随机当前值导致第一次计时不准。                  |